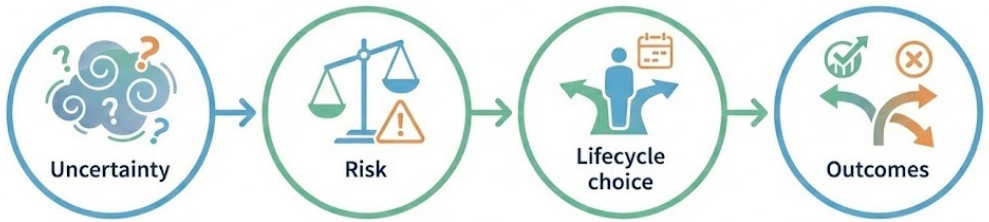
Methodologies

Why they exist, how to choose, where Agile breaks, and what AI changes

FinTech 512 – Software Engineering

Duke

Today’s Structure

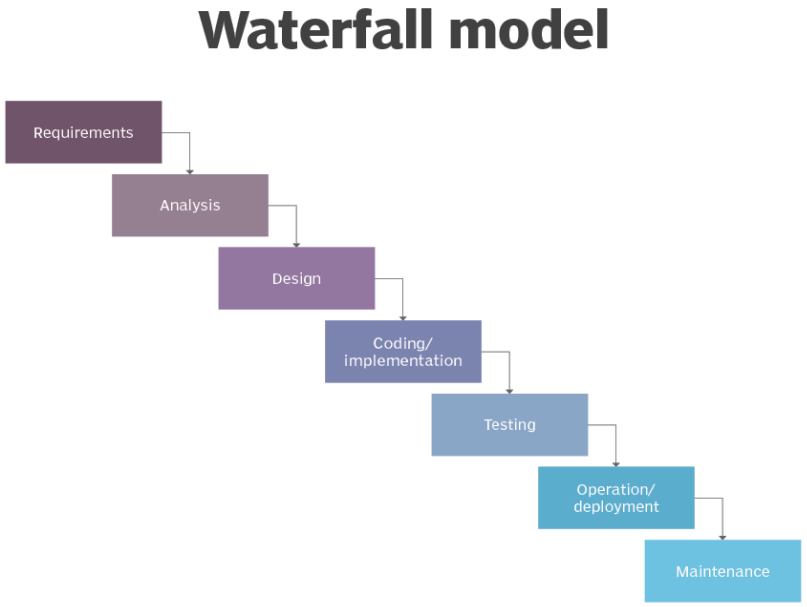


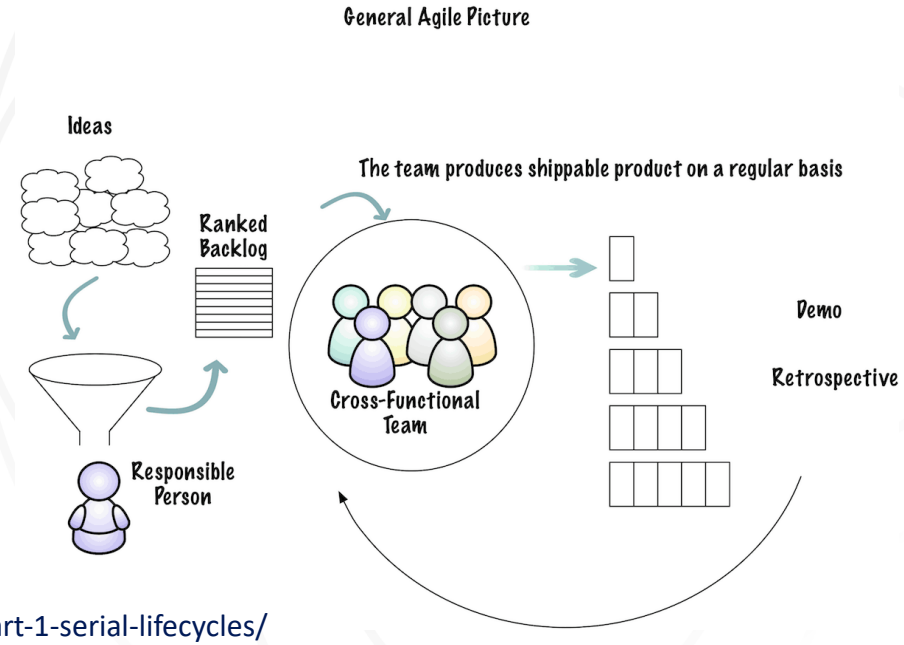
- Why methodologies exist
- Uncertainty types
- Risk profiles
- Lifecycle strategies as risk responses
- Choosing appropriately
- Where Agile breaks
- Hybrid reality in FinTech
- What AI changes

Duke

From Waterfall to Agile: Responses to Different Risks

- Uncertainty differs
- Decisions move
- Learning speed changes
- AI-accelerated





https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-1-serial-lifecycles/

Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

Individuals and interactions	over processes and tools
Working software	over comprehensive documentation
Customer collaboration	over contract negotiation
Responding to change	over following a plan

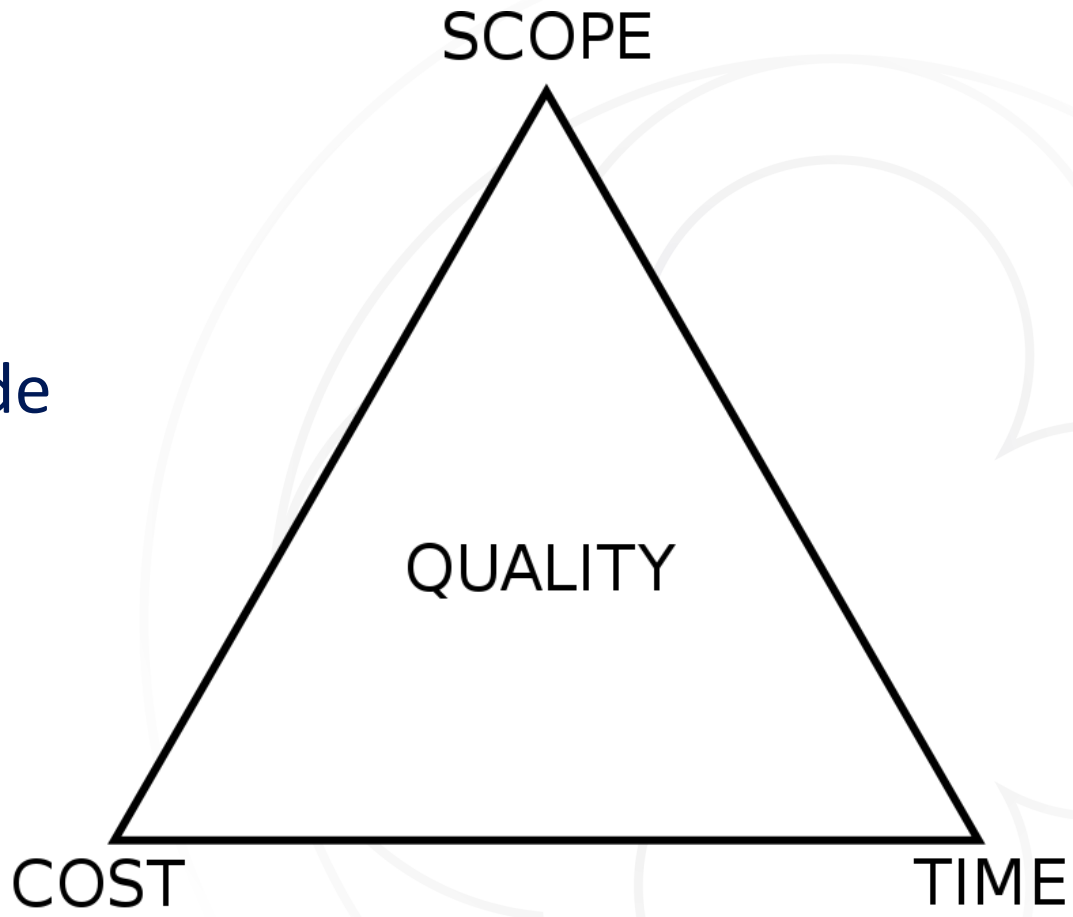
That is, while there is value in the items on the right, we value the items on the left more.

https://agilemanifesto.org/

Duke

Why Methodologies Exist: Constraints & Tradeoffs

- Constraints are real
- Tradeoffs are unavoidable
- Methodologies define how we trade



https://en.wikipedia.org/wiki/Project_management_triangle

But it’s more than a triangle

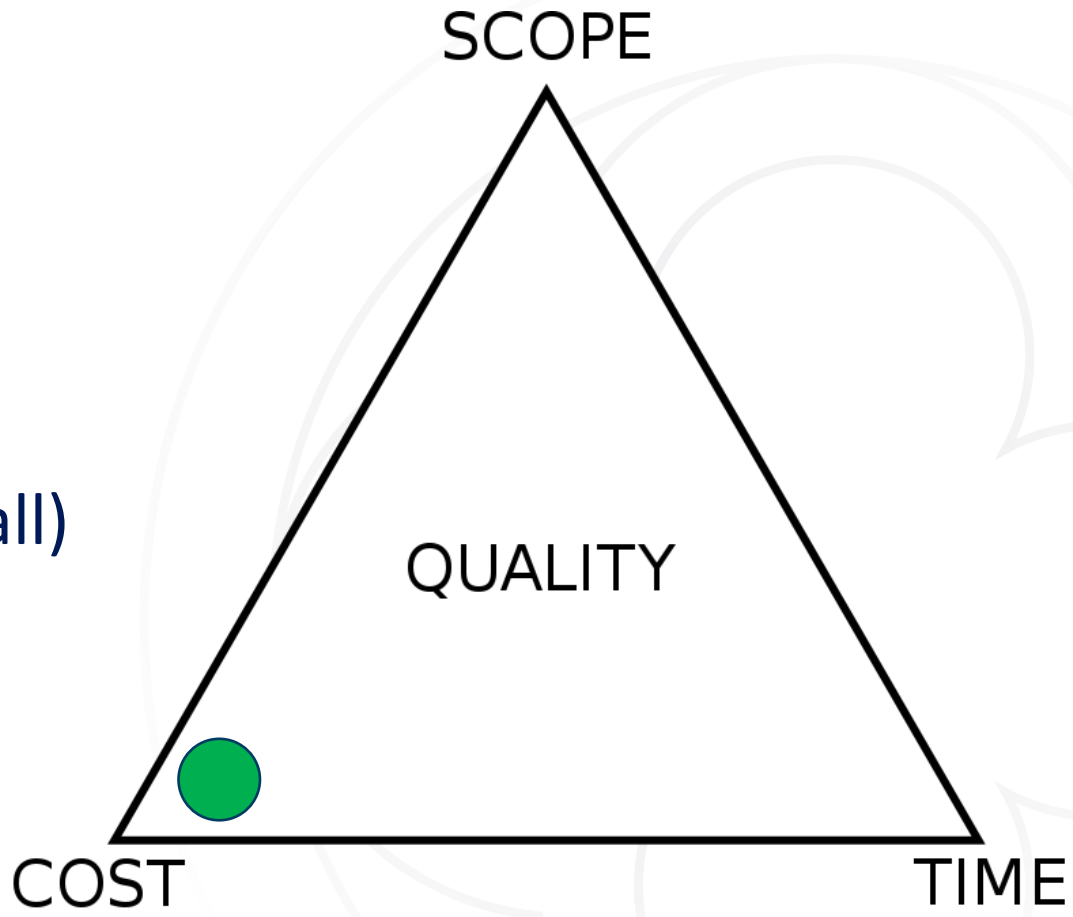
- Compliance
- Security
- Integration
- Operations
- Model risk (AI)
- Org / People

Duke

Serial Project Example: US Federal Tax System

- The triangle constraints mapping:
  - Scope: no flexibility (tax laws)
  - Time: no flexibility (April 15)
  - Cost: depends

Appropriate lifecycle: Serial (Waterfall)



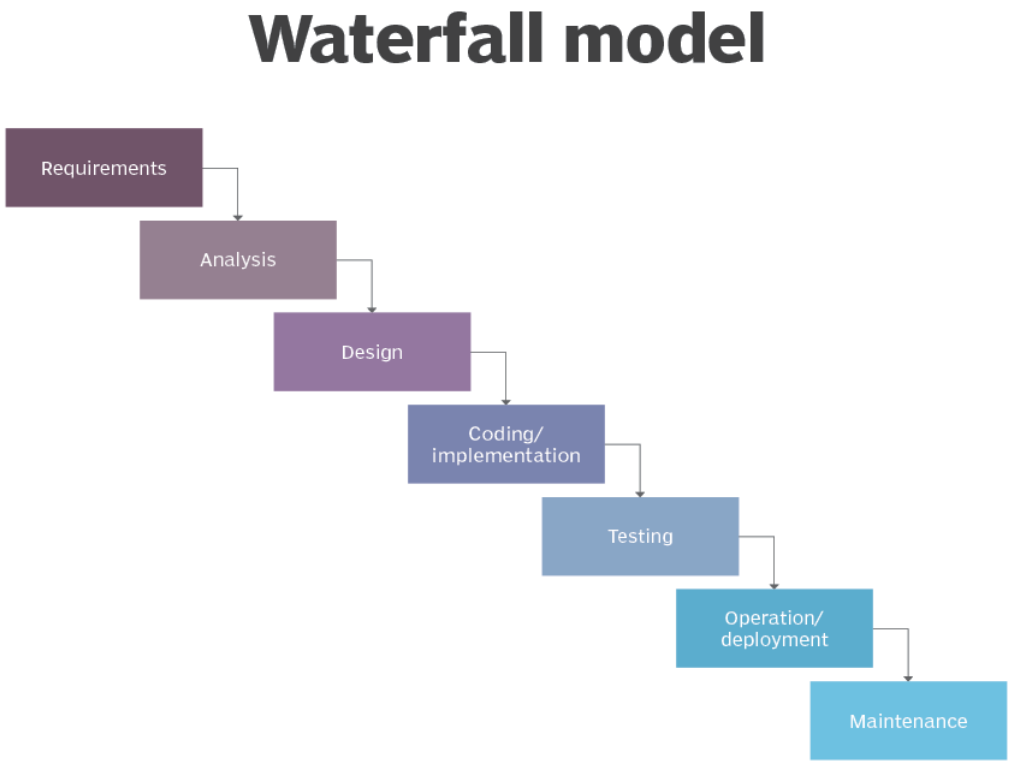
https://en.wikipedia.org/wiki/Project_management_triangle

Tax System Risks

- Requirements: tax code and forms are published in advance, low
- Schedule: missing April 15 the deadline, high
- Technical: need to understand own finances, tax forms, low-medium
- Regulatory/Compliance risk: high (errors/omissions → penalties, audit exposure)

Duke

Serial Lifecycle Walkthrough (Tax System)

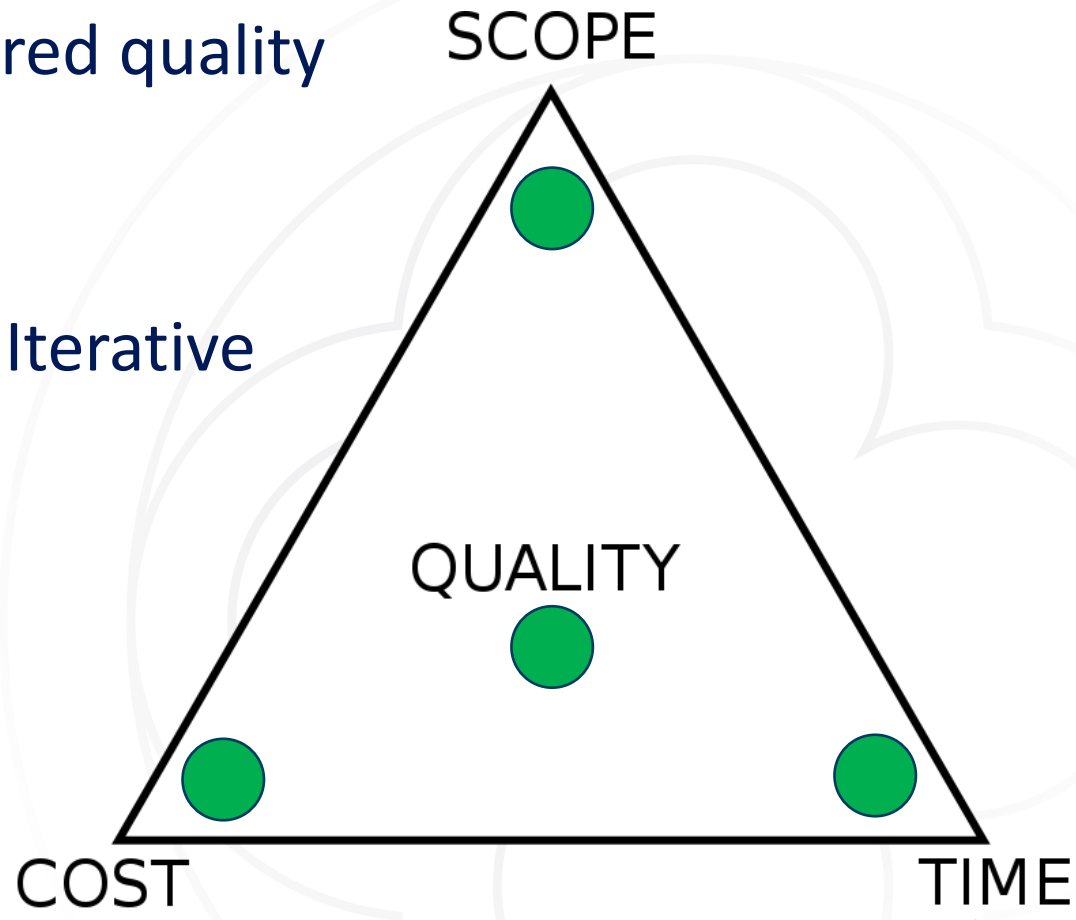


- Requirements: Tax code
- Analysis/Design: Government designs forms
- Implementation: People and organizations summarize finances, compare against forms, code, filling out forms
- Testing: checking work
- Release: File by April 15

Duke

Iterative Project Example: Learn to make your favorite meal

- Scope: Flexible – choose meal and desired quality
- Time: Flexible
- Cost: Flexible
- Appropriate project lifecycle approach: Iterative



Dominant uncertainty:
requirements + quality discovery

https://en.wikipedia.org/wiki/Project_management_triangle

Duke

Iterative Project Risks: Learn to make your favorite meal

- Requirements: for any given meal, a number of recipes of varying quality
- Schedule: take as long as you see fit
- Technical: may need to
  - learn new cooking techniques,
  - find the best ingredients,
  - obtain new kitchen equipment,
  - learn to distinguish good recipes from not-so-good
  - Find a mentor for some or all of the above

Duke

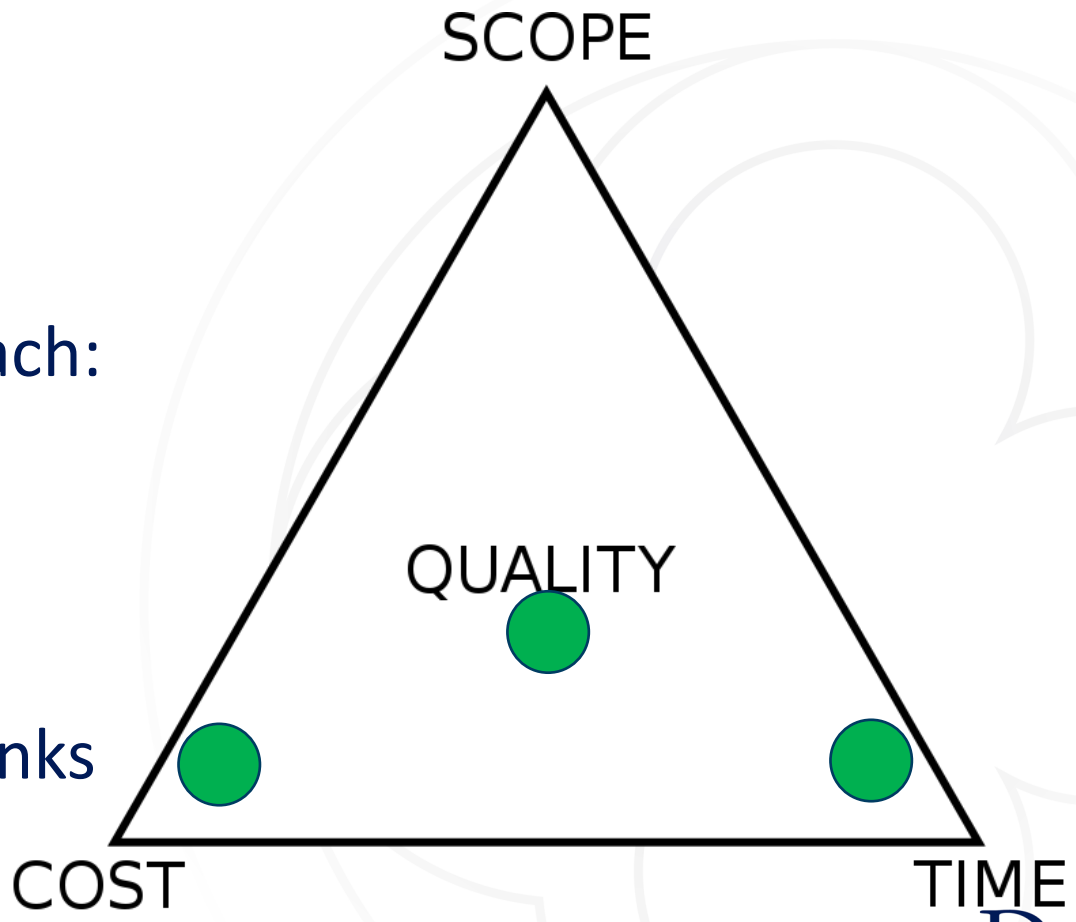
Iterative Lifecycle = Build → Learn → Adjust

- **Requirements**: Individual: ability to make your favorite meal
- **Analysis/Design**: Choosing recipes, ingredients, techniques
- **Implementation**: Making the meal
- **Testing**: tasting the cooked meal, revising based on how it went
- **Release**: Decide whether you've accomplished your goal or want to try again

Duke

Incremental Project Example: Find your three favorite restaurants in a new town

- Scope: Finite
- Time: Flexible
- Cost: Flexible
- Appropriate project lifecycle approach: Incremental



Dominant idea: deliver value in chunks (one restaurant at a time)

https://en.wikipedia.org/wiki/Project_management_triangle

Incremental Project Risks:
Find your three favorite restaurants in a new town

- Requirements: for any given own and your preferences, could be many options, low-medium
- Schedule: You can take as long as you see fit (until you move to another town), low-medium
- Cost: you may not be able to afford what would be your favorite restaurant or to eat out very often, low-high
- Technical: you know what you like, low

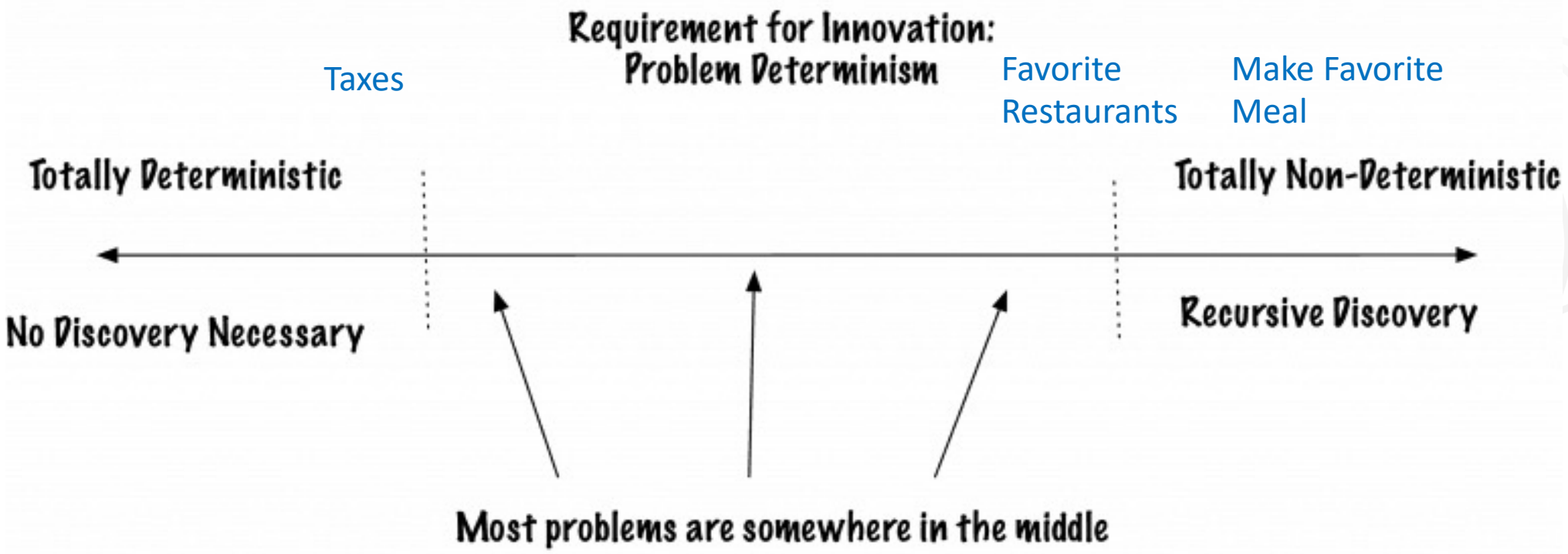
Incremental reduces risk by delivering value early

Duke

Incremental Lifecycle = add value in steps

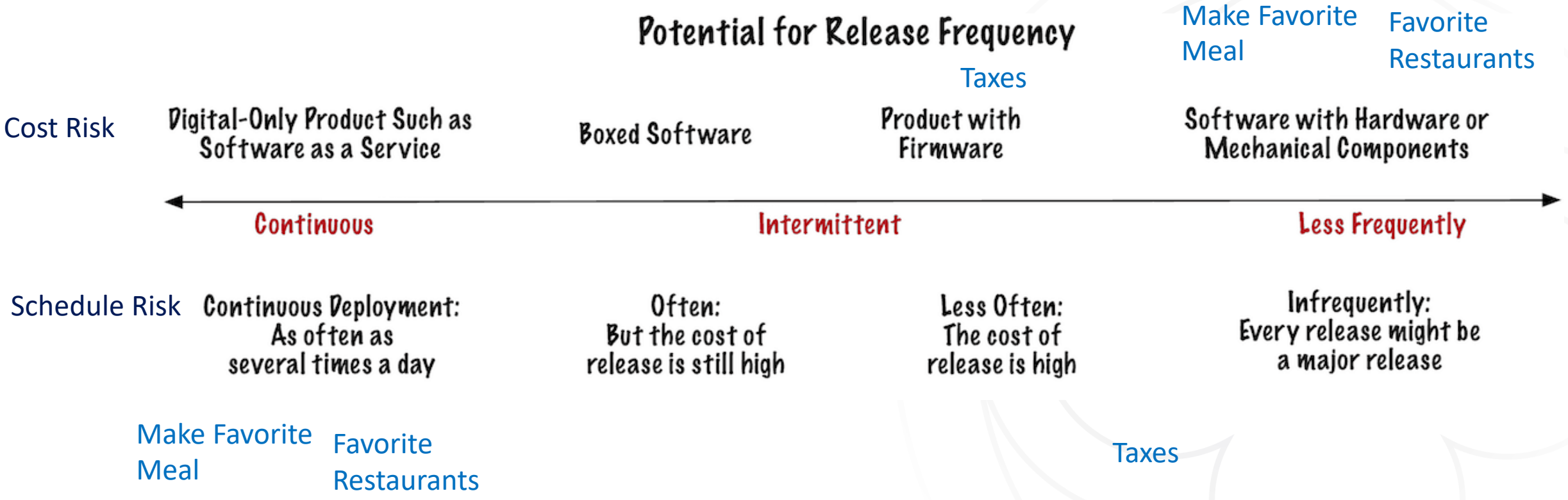
- Requirements: know your preferences and how to rank restaurants
- Analysis/Design: Choosing restaurants
- Implementation: Eat out
- Testing: decide whether you'd go back, add to your ranked list if so
- Release: Stop when you've tried as many restaurants as you would like

Requirements Risk



Duke

Schedule and Cost Risks



Duke

Technical Risks

- New Hardware
- New software
- New team
- …

Duke

Lifecycle strategies are risk responses

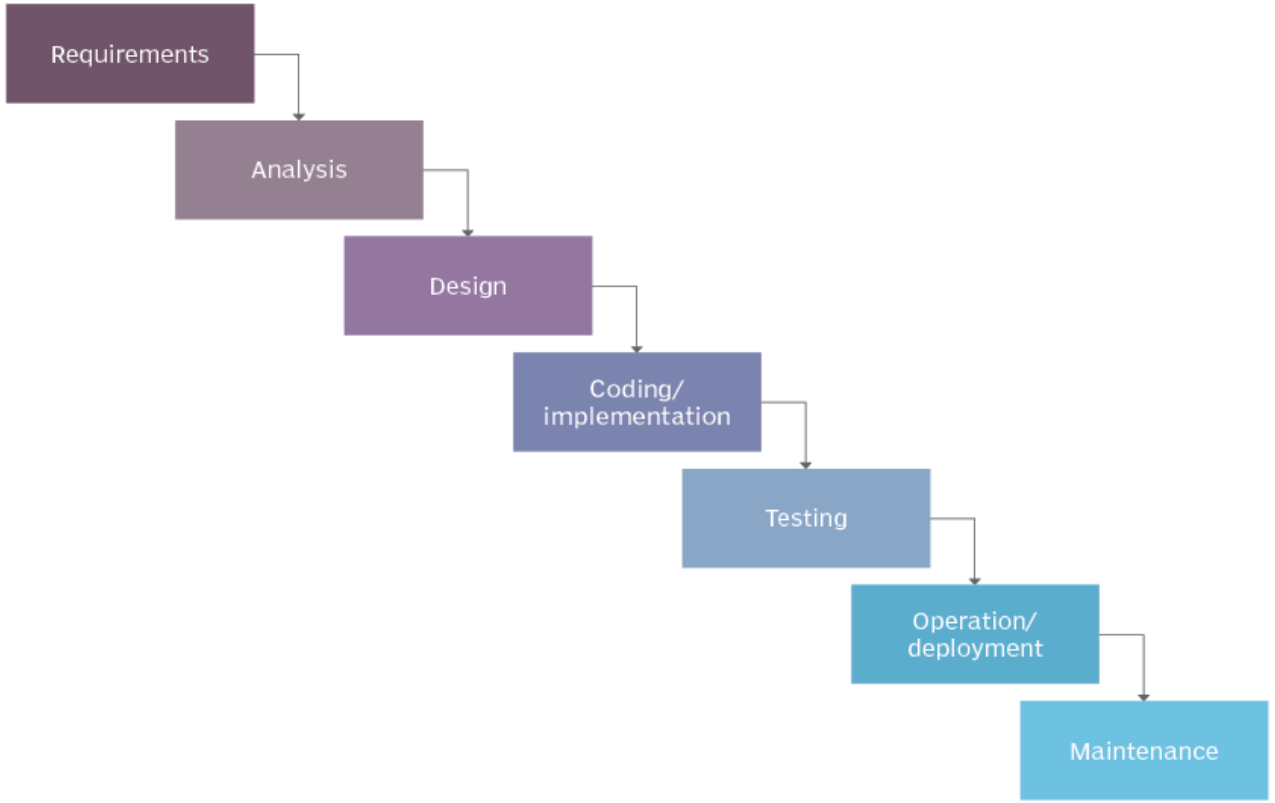
- **Serial** → control scope & compliance
- **Iterative** → burn down technical/requirements uncertainty
- **Incremental** → deliver value early, manage cost/schedule flex
- **Agile (iteration/flow)** → shorten feedback loops under change

Duke

Serial Lifecycles: Waterfall

Best fit when…

- Requirements relatively stable
- High compliance/audit needs
- Hard deadlines/release gates

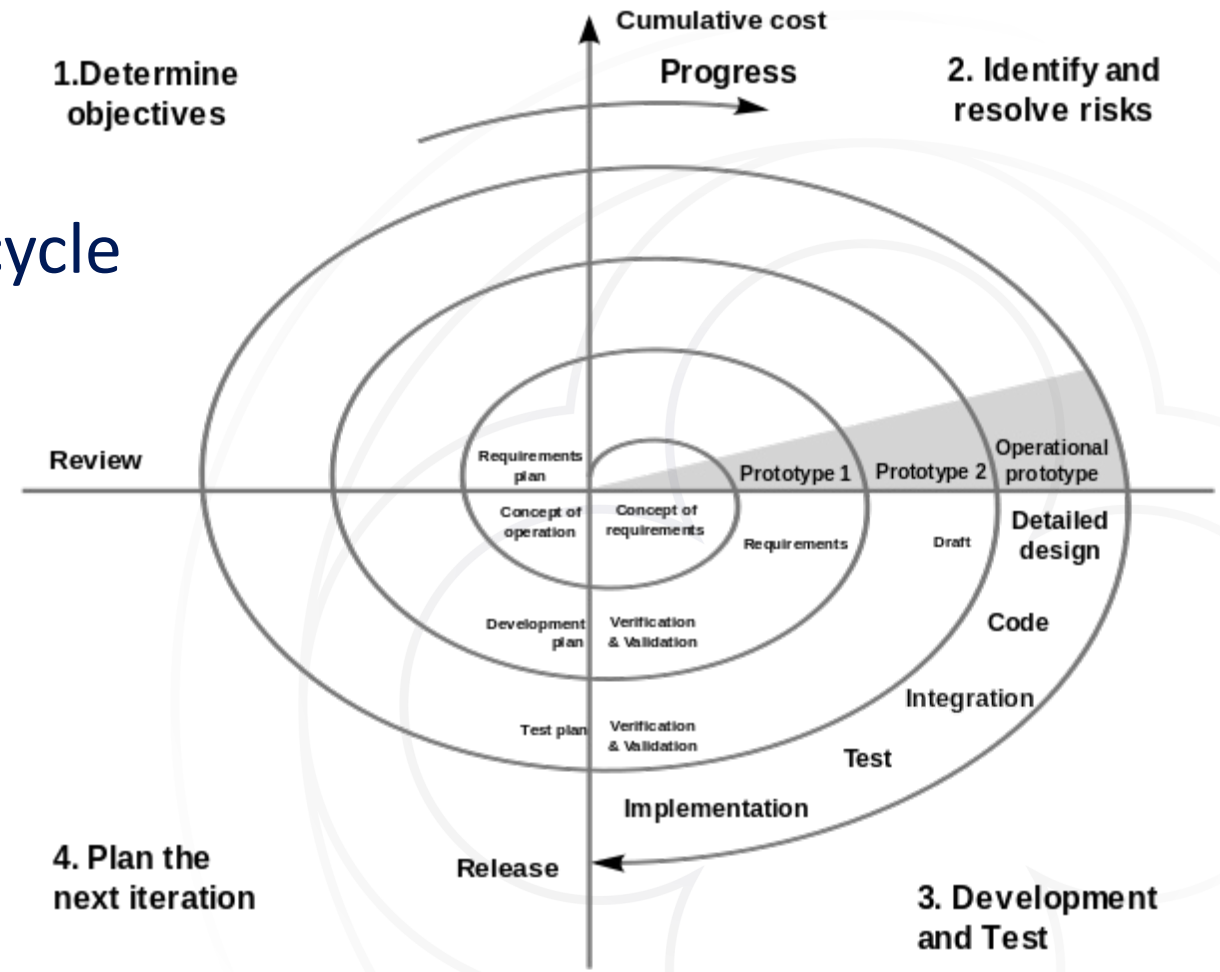


Duke

Iterative Lifecycles: Spiral

Purpose: reduce uncertainty each cycle

- Prototype
- Evaluate
- Decide



https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-2-iterative-lifecycles/

Duke

Iterative Lifecycles: Spiral

Spiral Life Cycle

Requirements	Prototype: Analysis, design, code. Check with customer. Replan based on feedback	Prototype: Analysis, design, code. Check with customer. Replan based on feedback	Prototype: Analysis, design, code. Check with customer. Replan based on feedback	... As many prototypes as necessary	Finish the prototypes: Complete the development work	Integration	Test

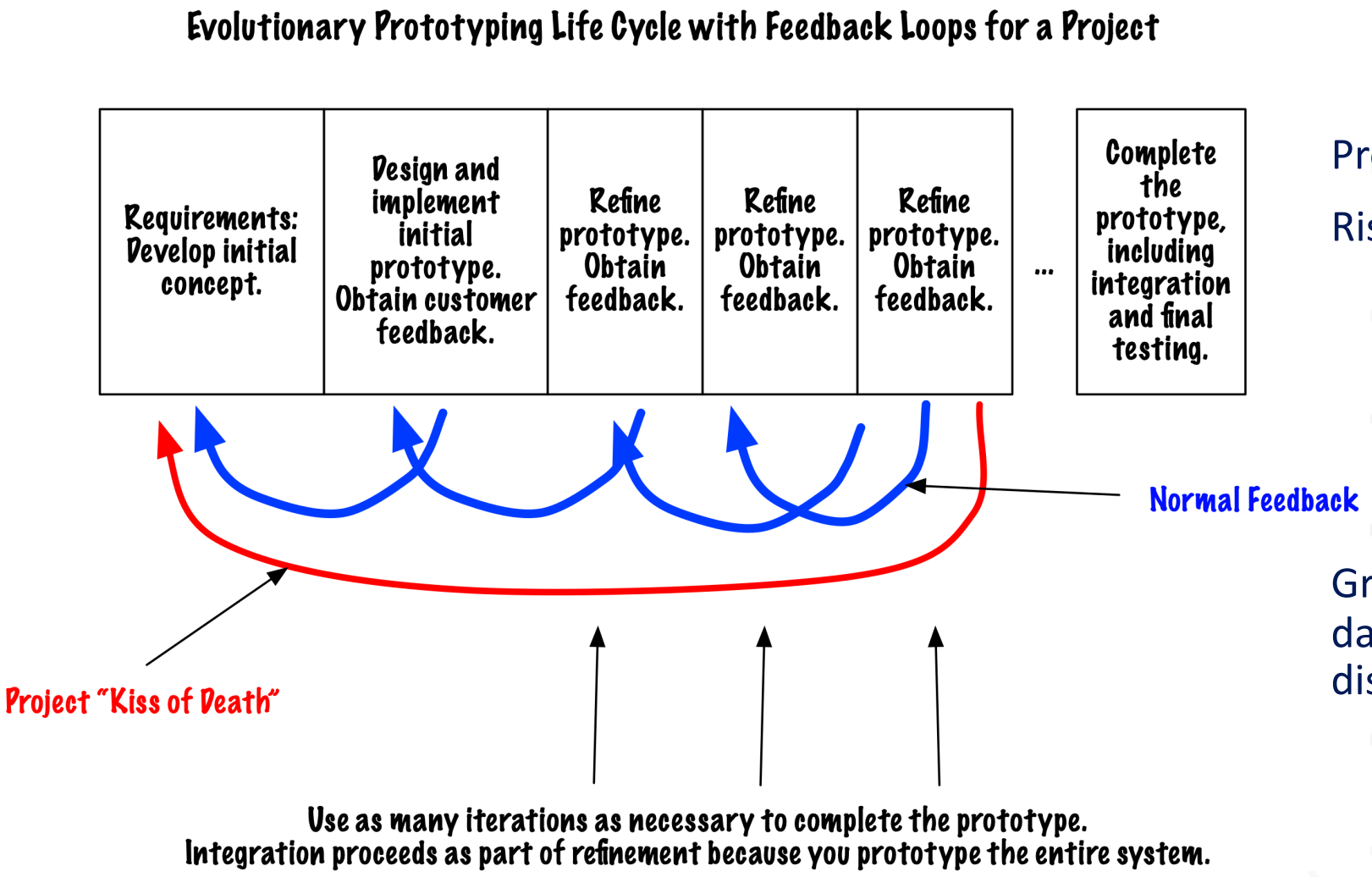
Note for times:
Each prototype spiral can be a different duration.
Remaining boxes not to scale with time.

Use as many iterations as necessary to prototype.
In a spiral life cycle, the prototypes do not need to be complete.

https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-2-iterative-lifecycles/

Duke

Iterative Lifecycles: Evolutionary Prototyping



Prototype is production-bound

Risk: prototype becomes the system

Great for requirements discovery; dangerous without engineering discipline.

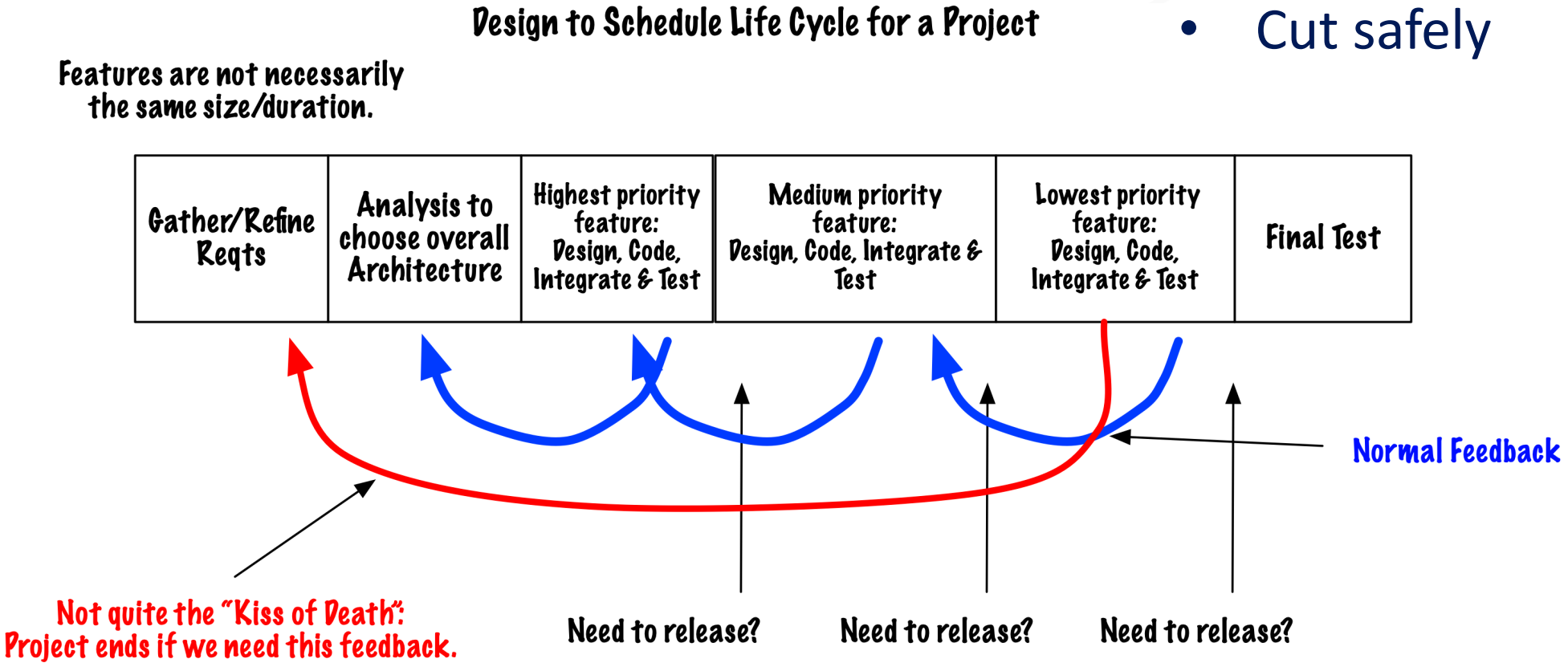
https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-2-iterative-lifecycles/

Duke

Incremental Lifecycles: Design To Schedule

Fixed date, flexible scope

- Prioritize by value
- Cut safely



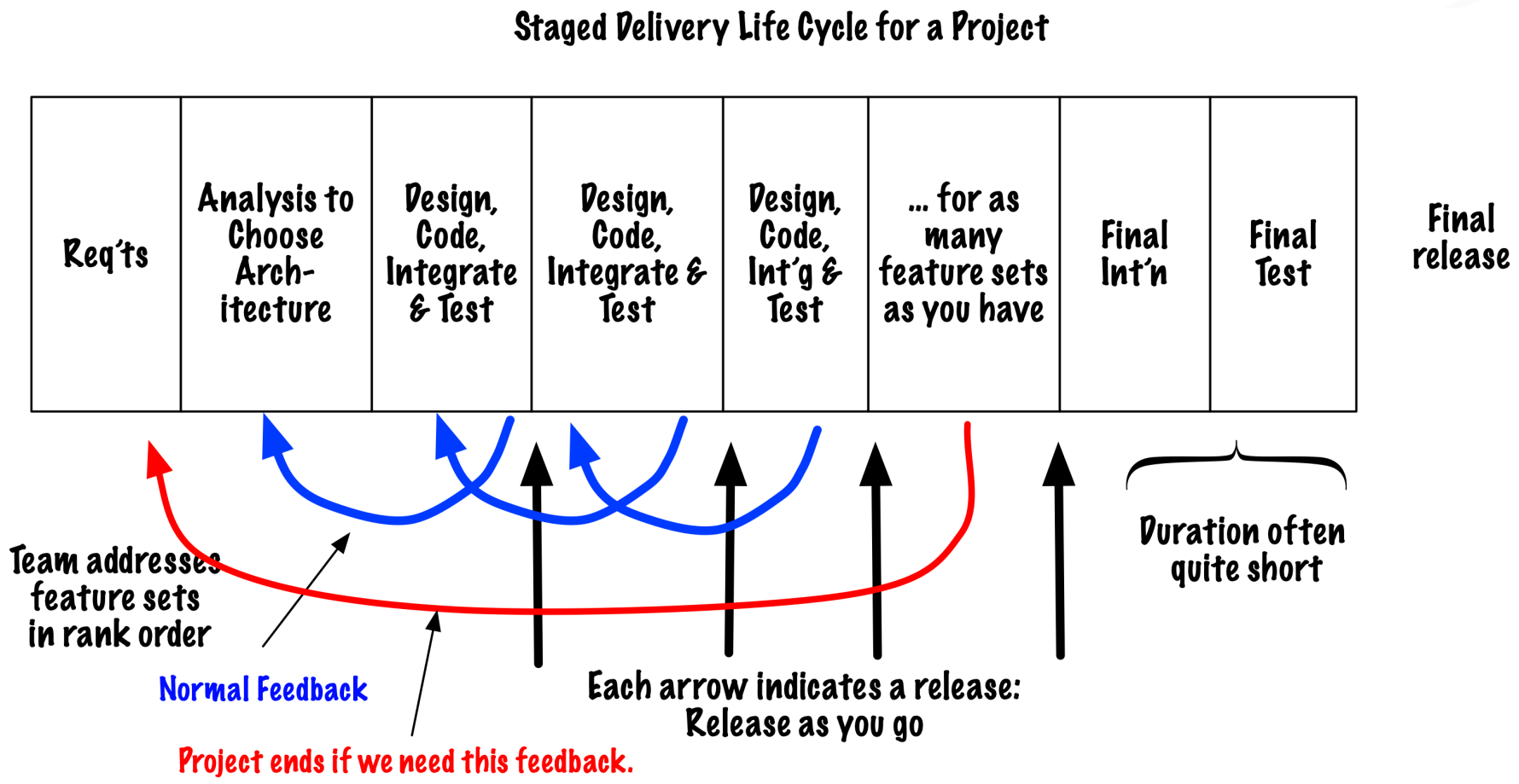
https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-3-incremental-lifecycles/

Duke

Incremental Lifecycles: Staged Delivery

Planned increments + validation gates

- Build
- Integrate
- Stabilize



https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-3-incremental-lifecycles/

Duke

Iteration-Based Agile

Timeboxed feedback loops

Iteration-Based Agile

Requirements Analysis Design Build Test Release Deploy	Requirements Analysis Design Build Test Release Deploy	Requirements Analysis Design Build Test Release Deploy	Requirements Analysis Design Build Test Release Deploy	Repeat as needed ...	Requirements Analysis Design Build Test Release Deploy	Requirements Analysis Design Build Test Release Deploy

Each timebox is the same size. Each timebox results in running tested features.

- Plan → Build → Review
- Regular cadence
- Reprioritize often

https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-5-origins-of-agile-approaches/

Duke

Flow-Based Agile

Flow-Based Agile

Feature: Clarify Req't, Analysis Design Build Test Release Deploy	Feature: Clarify Requirement, Analysis Design Build Test Release Deploy	Feature: Clarify Requirement, Analysis Design Build Test Release Deploy	Repeat as needed ...	Feature: Clarify Requirement, Analysis Design Build Test Release Deploy	Feature: Clarify Requirement, Analysis Design Build Test Release Deploy

In flow, the team limits the number of features active at any time with WIP limits for each team activity.
There is no timeboxing built into flow.

https://www.jrothman.com/mpd/project-management/2020/10/what-lifecycle-or-agile-approach-fits-your-context-part-5-origins-of-agile-approaches/

Duke

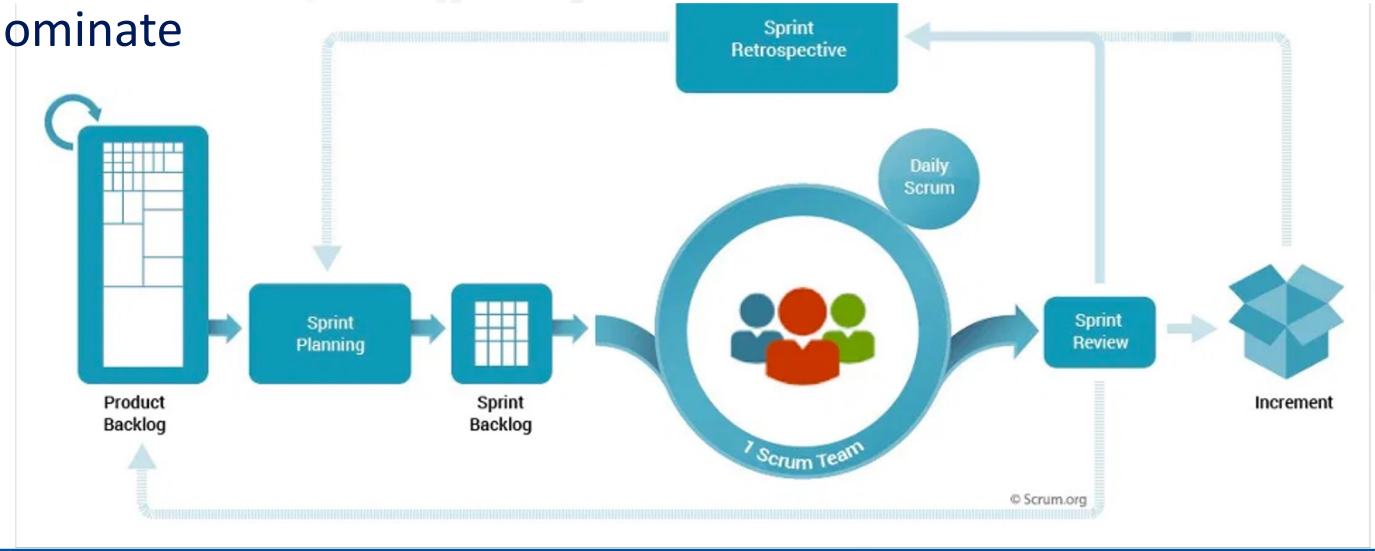
For what kind of projects is Scrum appropriate?

Scrum tends to work well when:

- Expect change and need fast feedback.
- Stakeholders can review frequently and make real decisions.
- ‘Done’ includes testing and integration, not just coding.

Scrum tends to struggle when:

- Compliance gates are slow and external approvals dominate.
- Can’t produce a shippable increment each sprint.
- Work arrives unpredictably and interrupts dominate



Choosing appropriately

A lifecycle selection model (not a taxonomy)

Lifecycle Selection Decision Model

Identify dominant uncertainty → Name primary risk constraints → Choose lifecycle strategy → Verify fit with real constraints

Common uncertainty types

- Requirements
- Technical
- Integration
- Regulatory
- Operational
- Organizational

Rule of thumb: pick the lifecycle that collapses your largest uncertainties earliest.

Duke

Selection Matrix

Dominant risk	Best-fit strategy	Common patterns
Hard deadline + compliance	Serial	Waterfall + stage gates; heavy evidence trails
Technical uncertainty	Iterative	Spiral; spikes; prototypes with exit criteria
Value in slices	Incremental	Staged delivery; design-to-schedule; feature flags
Constant reprioritization	Flow-based Agile	Kanban; WIP limits; cycle-time focus
Rapid feedback available	Iteration-based Agile	Scrum; sprint reviews; backlog refinement

Use this to start the conversation — then validate against real constraints and dependencies.

ke

Where Agile breaks

Failure modes, especially in regulated contexts

Agile breakpoints

Agile helps when it shortens feedback loops. It breaks when feedback cannot be acted on.

Governance lag

Approvals, audits, change control slower than delivery cadence

Architecture coupling

Hard to slice “shippable increments” safely

Dependency drag

Many external teams/systems → blocked work, stalled learning

Metrics gaming

Velocity theater replaces outcomes and risk reduction

Agile in FinTech: evidence, not paperwork

Agile values (keep)

- Short feedback loops
- Working software
- Respond to change

FinTech constraints (add)

- Audit trail + approvals
- Controls + segregation of duties
- Security + reliability

Good pattern: create evidence continuously (tests, logs, traceability) — not a giant document at the end.

Warning Signs: Agile Theater

If these are true, cadence won't create learning — it will create ritual.

- Sprint output isn't actually shippable (integration/testing deferred)
- Work is constantly blocked by external approvals or dependencies
- Velocity becomes the goal; outcomes and risk reduction are not measured
- Backlog changes daily; team cannot protect WIP or finish work
- Incidents/ops work overwhelms planned work (flow likely fits better)

Duke

Hybrid Reality

Most systems contain multiple risk profiles

Duke

Common Hybrid Patterns

Waterfall governance + Agile delivery

Stage gates for compliance; iterative delivery inside gates

Platform vs Product split

Platform runs flow (ops-heavy); product runs iterations (feature-heavy)

Staged rollout + feature flags

Incremental release to manage integration and operational risk

Hybrid ≠ chaos: define interfaces, decision rights, and evidence requirements across teams.

Duke

FinTech mapping: subsystem → lifecycle fit

Subsystem	Dominant risks	Good-fit lifecycle
Core ledger / accounting	Correctness, auditability, change control	Serial governance + staged delivery
Payments / transfers	Integration + operational reliability	Incremental + staged rollout
Fraud models	Concept drift, data quality, monitoring	Iterative + tight eval loops
Customer UI	Requirements change, usability feedback	Iteration-based Agile
Ops / incident work	Interrupt-driven, urgent fixes	Flow-based Agile (Kanban)

AI-Driven Methodologies

What’s Changing and What isn’t Changing …

Duke

AI Changes the Economics of Development

AI reduces the cost of creating code. It increases the importance of verification and governance.

AI accelerates

- Scaffolding & refactors
- Test drafting
- Prototyping & spikes
- Docs/review assistance

AI increases risk in

- Plausible-but-wrong code
- Security & data handling
- Hidden dependencies
- Auditability gaps

AI Does Not C

- Compliance requirements remain
- Operational risk remains
- Accountability remains
- Systems still fail at integration boundaries

Duke

AI Era Loop: Spec → Generate → Verify

As coding gets cheaper, methodology shifts toward verification discipline.

Spec
Define behavior, constraints, acceptance tests

→

Generate
AI-assisted implementation + refactor

→

Verify
Tests, security checks, reviews, evidence

→

Operate
Monitor, rollback, audit trail, incident response

Guardrails (FinTech)

- Define “done” as evidence (tests + controls)
- Record provenance (what changed, why, who approved)
- Treat model/code suggestions as untrusted input

Duke

Key Takeaways

- Methodology is a risk strategy (not an identity)
- Choose based on dominant uncertainty + real constraints
- FinTech is hybrid by default: compliance + operations matter
- AI speeds building; verification and governance become the bottleneck